

Applied Data Science on Customer Retention Strategy: Predictive Modelling for Churn Detection in E-commerce

Name: Silvana Hatsue Koga Alarcon

Date: 19/09/2026

Index

Problem set	3
Introduction	3
Brazilian sector	3
Churn detection	4
Strategic approaches for customer retention	4
Data presentation	5
Exploratory Data Analysis (EDA)	5
Data Structure	5
Statistical summaries	6
Data Preparation	14
Data cleaning	14
Data splitting criteria	15
Feature engineering	15
Data Modeling	17
Hypothesis testing	17
Logistic Regression	18
Decision Tree	19
Random Forest	19
XGBoost	20
Evaluating Results	20
Goodness of fit testing(H1)	20
Evaluating the second hypothesis	21
Business conclusions and Insight	21
Bibliography	22
Appendix	23

Problem set

The business department asked for a churn prediction model that improves marketing decision-making. The marketing team is responsible for sending promotional emails to customers who are likely to stop buying on the platform. The mission is to study consumers' shopping patterns and find the optimal threshold day to detect customers who could churn with high probability using machine learning.

To achieve and create the churn detection model, we will use the Brazilian e-commerce public dataset created by Olist, a B2B2C marketplace enabler that helps small and medium businesses to promote and sell their products on the Olist Shop to compete with dominant e-commerce platforms in Brazil, such as Mercado Livre, Amazon, Americanas, and Shopee. The Olist e-commerce dataset covers customer purchases between 2016 and 2018.

Introduction

Since the introduction of the internet, people have seen different styles of online businesses as the years passed. It started with the National Science Foundation selling of domains, and then restaurants started delivery services for their customers located within the service area. Later, social media platforms appeared to connect people, and businesses began using these sites as their sales intermediaries where they could promote their products to a wider population.

While today thousands of Internet shopping sites have been created, most buyers concentrate on marketplaces including Amazon, Rakuten, and eBay.

Brazilian sector

The Brazilian e-commerce sector has followed a parallel growth with other advanced economies. The continuous growth in this sector is due to three essential consumer protection laws that were created to safeguard Brazilians' product and service consumption on the internet.

The first critical law for consumer protection was enacted in 1990, called the Consumer Protection Code (CDC). The code encompasses specific regulations for e-commerce that ensure consumers get the correct information and privacy and provide consumers with the right to desist from their purchases within seven days.

Although the government provided basic rights to buyers with the CDC, it did not consider modern shopping on the Internet. Therefore, to complement the Consumer Protection Code (CDC), the Brazilian government established the E-commerce Decree in 2013 that requires online shopping platforms to display transparent information about their products and services, as well as payment methods.

Finally, to protect Brazilian citizens' safety on the internet, the Internet Civil Rights Act was introduced, promoting protection of privacy and fair data treatment for all individuals.

The Brazilian e-commerce market witnessed gradual growth from 2016, a year in which Brazil started to recover from a two-year recession caused by a decrease in domestic variables and a poor allocation of public credits emitted by the central bank. In 2017, dominant internet shopping platforms like Americanas, Magazine Luiza, and Via Varejo started turning their business model from selling products that they demand from wholesalers (1P model) to allowing third parties (merchants) to sell their products on the prestigious platforms (3P model). The emergence of the third-party (3P) business model in the Brazilian e-commerce industry influenced the creation of marketplace enablers that assist medium and small producers lacking advertising and logistics capabilities to sell their products on these marketplace enablers' shopping websites.

The numbers demonstrate how previous events expanded the market. In 2016, the total e-business revenue amounted to 44.4 billion Brazilian Real. The figure increased in 2017, reaching 50 billion, and jumped to 70 billion in 2018.

Churn detection

Before proceeding, it is critical to define the technical term "churn" to clarify the core objective of this study and to examine how other enterprises leverage this metric to optimise their business strategies.

First, when searching for the meaning of the term "churn" on the Cambridge Dictionary, we obtain two main explanations:

- **"A large container to make milk into butter by mixing it with great force"**
- **"The situation in which customers stop buying the products or services of a particular company, especially to buy them from a competitor."**

The second explanation closely matches our definition of churn, representing what is known as "active churn," where a client actively decides to stop buying. However, to be more precise with the definition, we need to distinguish between contractual churn and non-contractual churn.

Contractual churn refers to a specific situation where a consumer was under a contract to receive products and services from the business, but they actively decided to terminate the contract. In contrast, in non-contractual churn, customers just cease purchasing without any notification, since they did not sign any contract.

Considering the nature of most online buying businesses and Olist selling platforms, the target of this paper would be to detect non-contractual churn and discover a cutoff period (in days) that helps maximise the prediction of our investigation.

Strategic approaches for customer retention

When it comes to retaining consumers, most would think of techniques such as increasing advertising and marketing to the population or measuring customer satisfaction. However, in contemporary society, market conditions are constantly changing due to the introduction of AI and automated workflows, and enterprises are forced to reduce costs and improve agility in decision-making to adapt to the new context.

In order to elevate marketing efficiency and ensure client retention, some firms use the "threshold method." This method consists of analysing historical purchase data to determine the average time between purchases and calculate a window (e.g., 180 days). If the buyer does not purchase within this timeframe, they will be considered churned.

After defining the correct observation window for the commerce, they use AI-powered machine learning to automatically notify the marketing and advertising department of "at-risk-to-churn" buyers. Once the personnel are notified, they can engage with that at-risk-to-churn customer by sending advertising emails or discount vouchers to invite them to purchase products.

Data presentation

As mentioned before, a dataset provided by Olist, the Brazilian marketplace enabler. This public data source is available on Kaggle and will be explored and used in machine learning to build a model that reveals the characteristics of a customer about to churn, and the appropriate window to detect churn.

The dataset comprises 100,000 orders, allocated in one fact table that registers business events (orders), and eight dimension tables that store specific transaction details between 2016 and 2018.

Dimension tables provide the following information:

- Customer details
- Spatial data
- Order items specification
- Payment methods
- Customer review
- Products characteristics
- Sellers and their location
- Product categories

To improve data collection usability in the study, the fact and dimension tables are merged to generate a single dataframe and proceed with Exploratory Data Analysis (EDA). The dataframe is called "Olist_df", and the merging process is visible in the Appendix.

Exploratory Data Analysis (EDA)

In this section, we will inspect the Olist dataframe characteristics to understand its structure and describe Olist business performance using the Pandas library in Python.

Data Structure

Dataset overview

The Python program output that describes the olist_df structure is shown below.

Shape	(119143, 40)
string	24
float	15
integer	1

The dataset consists of 119,143 rows and 40 columns. When analysing the data types, we note that 24 variables contain text values, 15 include floats, and only one has an integer value.

Missing Values

We also explore null values (see Appendix), and the output displays different null values for each column. It's evident that "review_comment_title" and "review_comment_message" have a high number of null values. If we calculate the null ratio for these two columns, "review_comment_title" gets 88.3 and "review_comment_message" gets 57.8 %. These two features are candidates to drop since more than half of their values are null, and they can not provide valuable information for the study.

Numerical/Categorical Data

The unique function is the most direct approach to differentiate between categorical and numerical data. This function displays unique values for each column, and since categorical variables depict group characteristics, they will have fewer unique values.

According to the program output (see appendix), the categorical features are the ones listed in the table, and the rest are numerical and timestamp data.

customer_state	order_status	seller_city
payment_type	seller_state	customer_zip_code_prefix
seller_zip_code_prefix	product_category_name_english	product_category_name
customer_id	customer_unique_id	order_id
order_item_id	product_id	seller_id
review_id	review_comment_title	review_comment_message

Statistical summaries

This section presents a statistical summary of the quantitative, qualitative, and timestamp features to gain further information about the dataset.

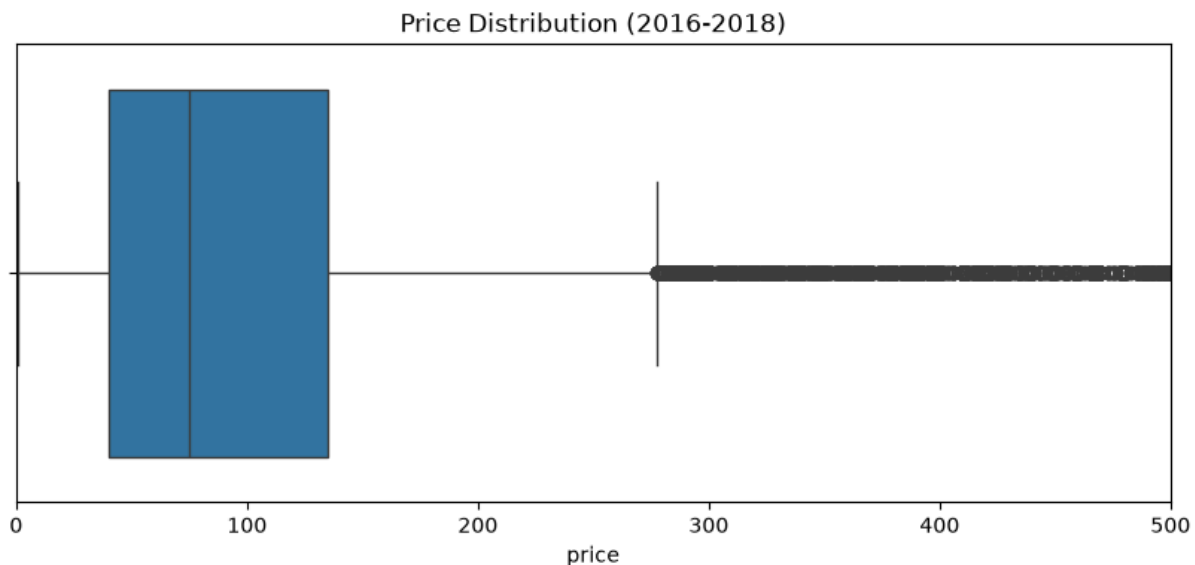
Quantitative variables

price

The minimum price of a product is 0.85 Brazilian Reais, while the maximum is 6,735 Brazilian Reais(Appendix Table 1.1). It is notable that the price is positively skewed, as the mean is influenced by the maximum price and remains greater than the median. Regarding the variance, the coefficient of variation (CV) is 152.6%, indicating that variation is extremely high.

The price distribution can be appreciated in the following graph, with a limit on the x-axis.

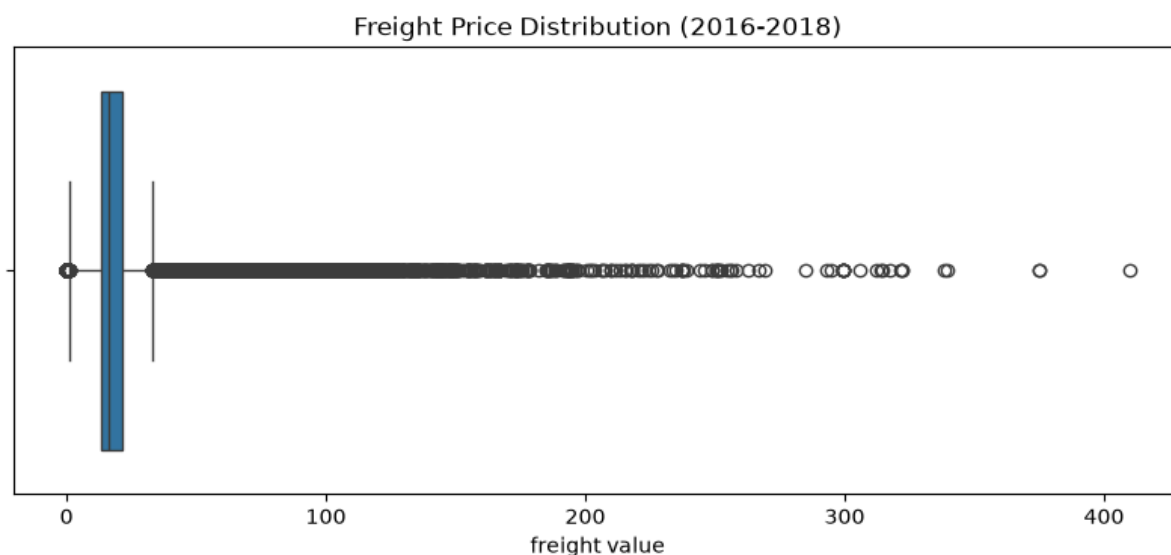
Graph 1.1



freight_value

For freight value, the minimum value is 0 Brazilian Reais, and the maximum is 409 Brazilian Reais. This attribute is also positively skewed, but contrary to price, its CV is around 79.06%, which is below 100%. Therefore, it can be stated that there is not an extremely high variation.

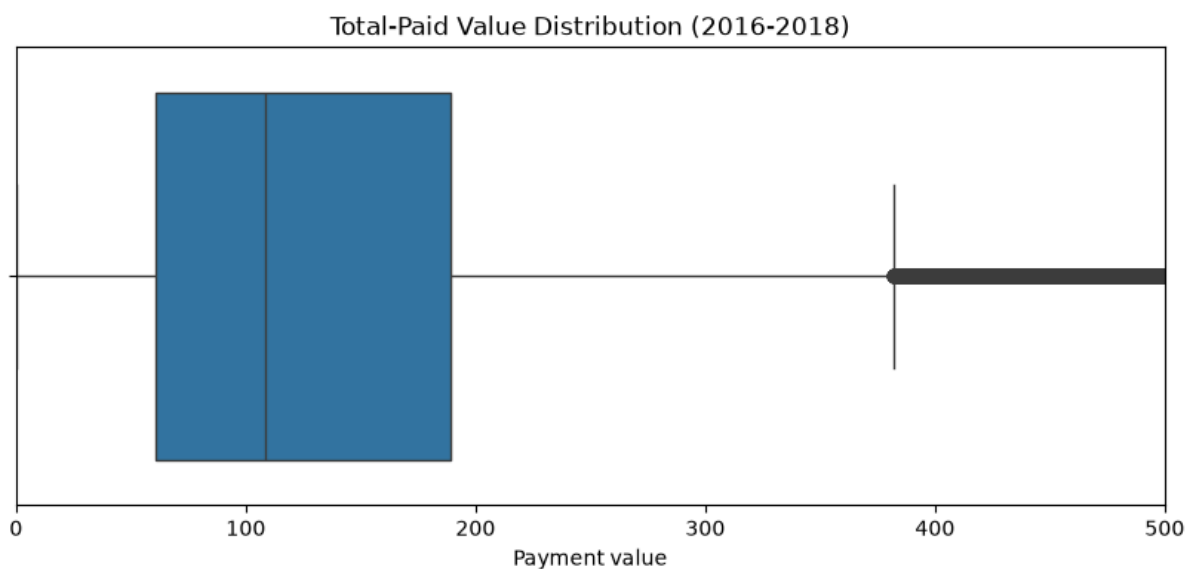
Graph 1.2



payment_value

Payment value ranges between 0 and 13664.08 Brazilian Reais. The mean and median numbers show positive skewness, while on the other hand the coefficient of variation is high (155.02%), which is a sign of significant variance.

Graph 1.3



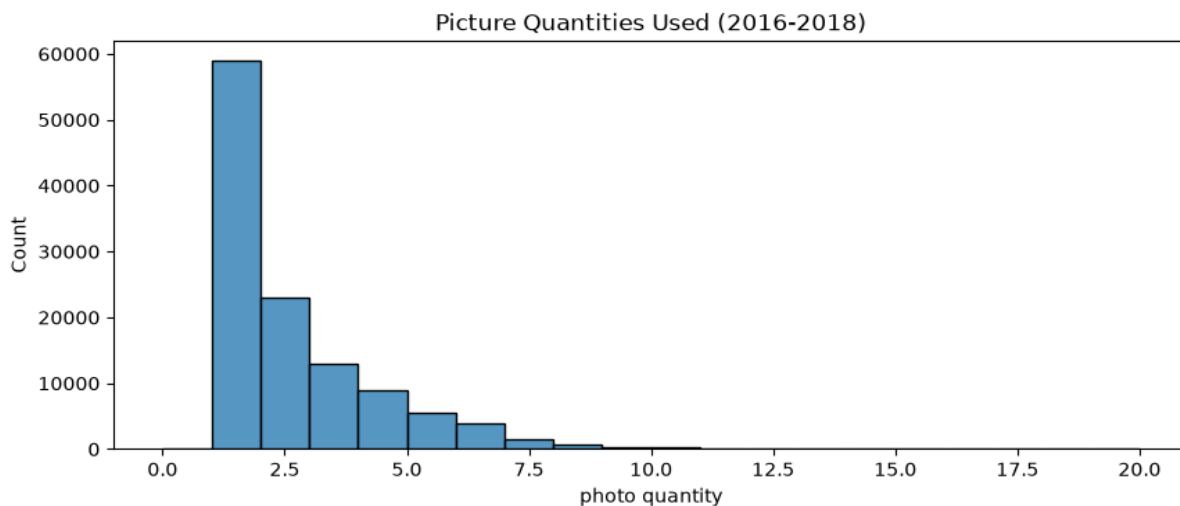
product_name_length

The values in this column are distributed between 5 characters and 76 characters. In this case, we have a slight negative skewness, as the difference between the mean and the median is -7.69%. With a CV of 20.57%, we can express a moderate variation in name length.

Product presentation (description length, photo quantity)

Product descriptions range from 4 to 3992 letters, while products have between 1 and 20 pictures. Both product description length and photo quantity have minimum positive skewness, and they present high variance.

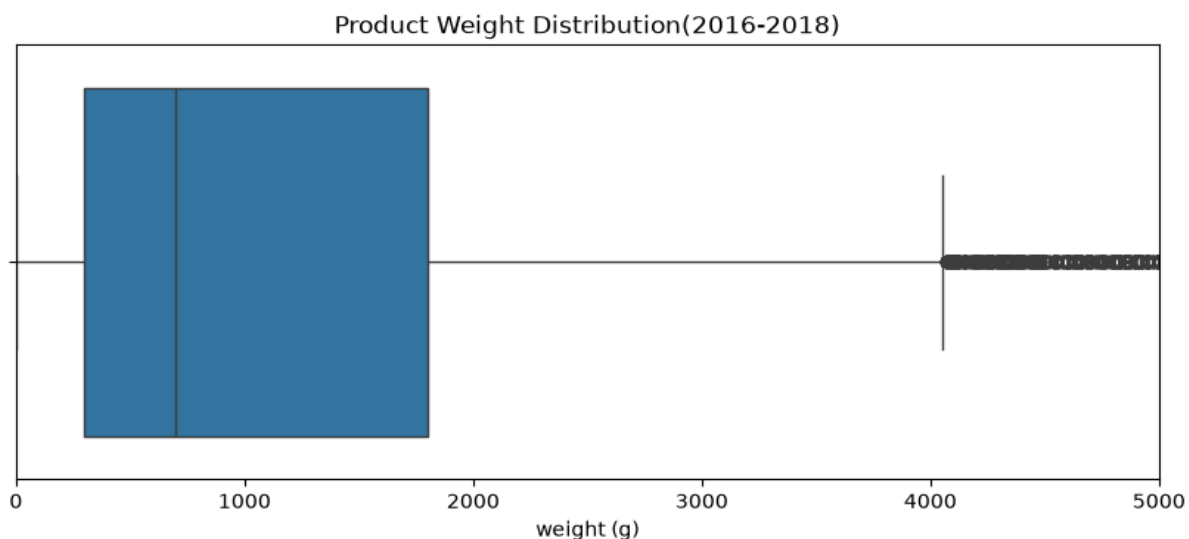
Graph 1.4



Product dimensions (weight, length, height, and width)

Length, height, and width reveal a slight positive skewness and wide dispersion. However, weight manifests pronounced right-skewness, registering a much higher CV.

Graph 1.5



On average, individuals use about one payment split("payment_sequential"), but 29 was the maximum number counted. In terms of the payment periods used to finish purchasing the product, the average customer used around 2 months, while some used up to 24 months.

Furthermore, review score situate between 1(worst) and 5(best), with the average purchaser leaving a score of 4.

Qualitative variables

According to Table 1.4 in the appendix, the most frequent place appearing for customer state and city is São Paulo. Clients tend to use credit cards as a payment method, and the positive words "recommend" and "very good" appear frequently in comments.

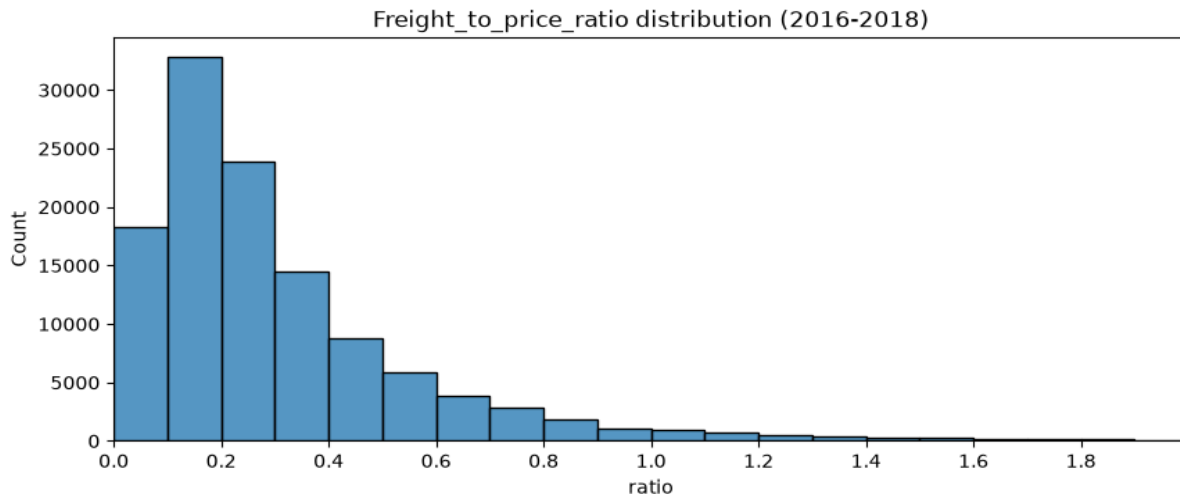
As Table 1.5 illustrates, sellers located in São Paulo are predominant, and the Olist top-selling categories are bed, bath, and table.

Business Metrics

Freight ratios

Calculating the proportion of a product price consumed by freight cost is essential to detect exorbitant transportation expenses that may discourage a customer from completing the purchase or from returning to purchase again (non-contractual churn). The graph shows the distribution of the freight-to-price ratio.

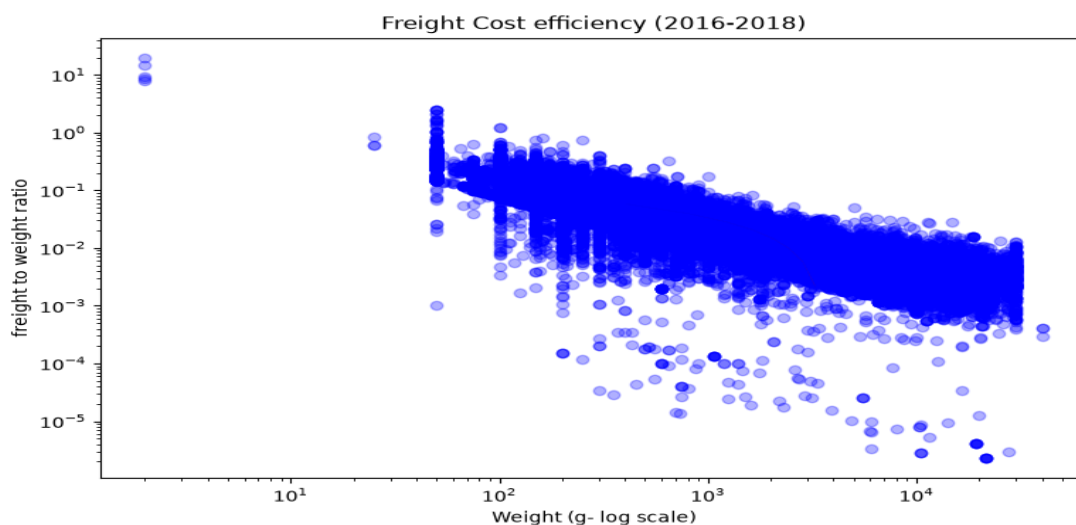
Graph 1.6.



Detect). As illustrated in Graph 1.6, the most frequent rate is between 10 and 20% with respect to price, which is slightly high. The problem is that the second most common ratio is above 20%. A transportation cost exceeding 20% is considered extremely high, and often it signifies a poor tariff-reduction strategy or freight cube utilisation that increases logistics expenses.

To investigate if logistic inefficiency exists, we have formed a log-scaled scatter plot that reveals weight and freight-to-weight ratio

Graph 1.7.

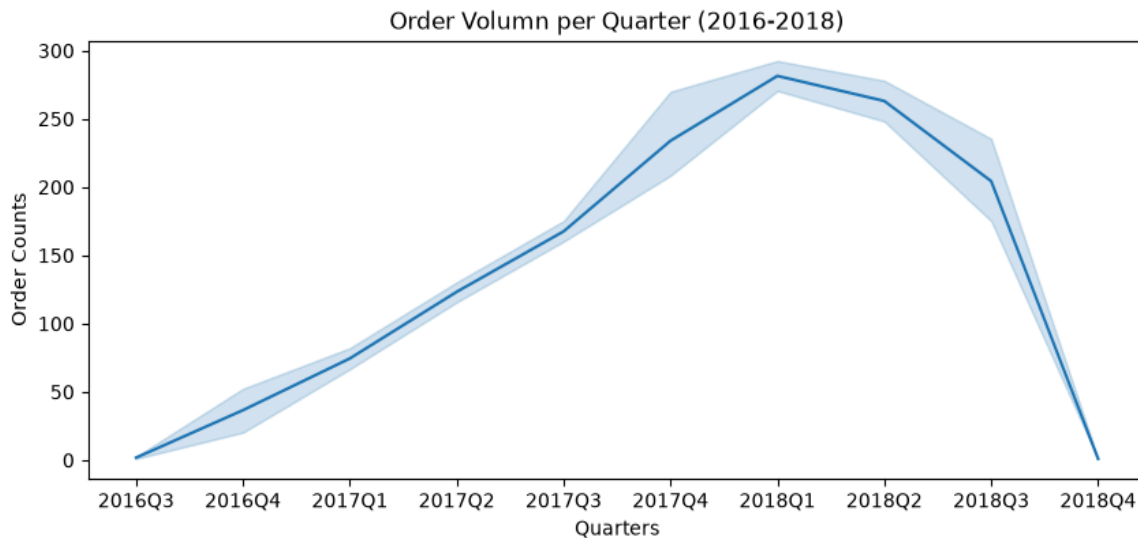


The graph exhibits a negative relationship between freight-to-weight ratio and weight. These downward dots show that lighter products will have relatively high costs. Nevertheless, this plot reflects economies of scale, where an increase in product weight reduces freight cost efficiency.

Selling volume per quarter

Sales volume per quarter can capture seasonal trends, allowing businesses to identify factors that drive shopping. Although this trend is not confirmed in the visualisation below, we can say that Olist quarterly sales registered low volume in 2016; it starts going upward from 2017Q1. In fact, the Olist company was launched in 2014 and officially started operating in 2015. This explains why the firm experienced a steep growth in orders.

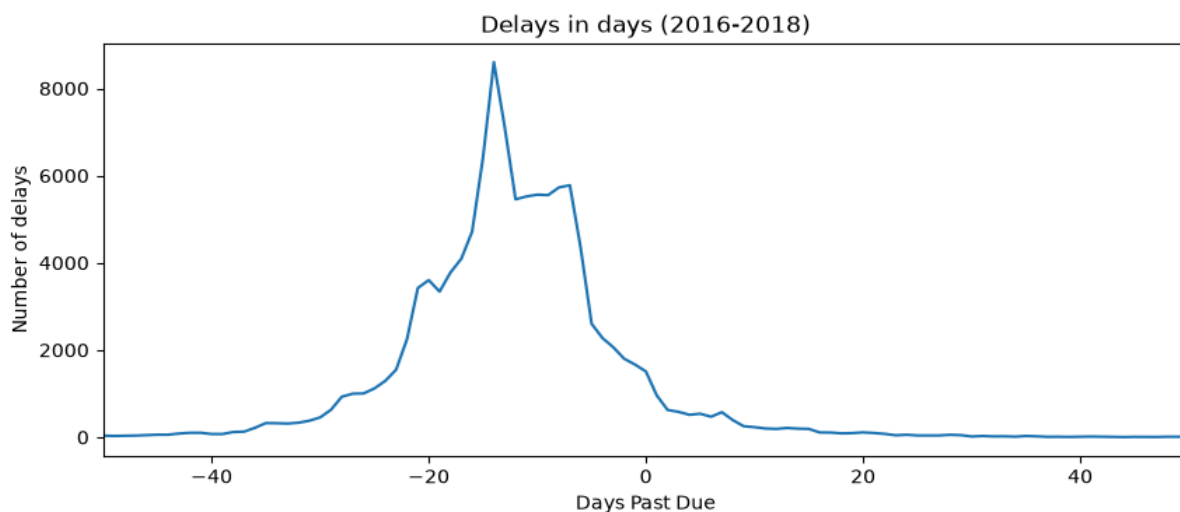
Graph 1.8.



Delivery efficiency

Herein, we will examine the company's delivery performance. The Python code will build a line visualisation describing whether there were delays or whether some products arrived early. To compute the shipping delay, the date the customer received the product at their order location was subtracted from the estimated shipping arrival date. Therefore, positive values are regarded as delays, while negative values point to early arrivals. The visualisation shows only a specific range of x, but the full version can be seen in the Appendix.

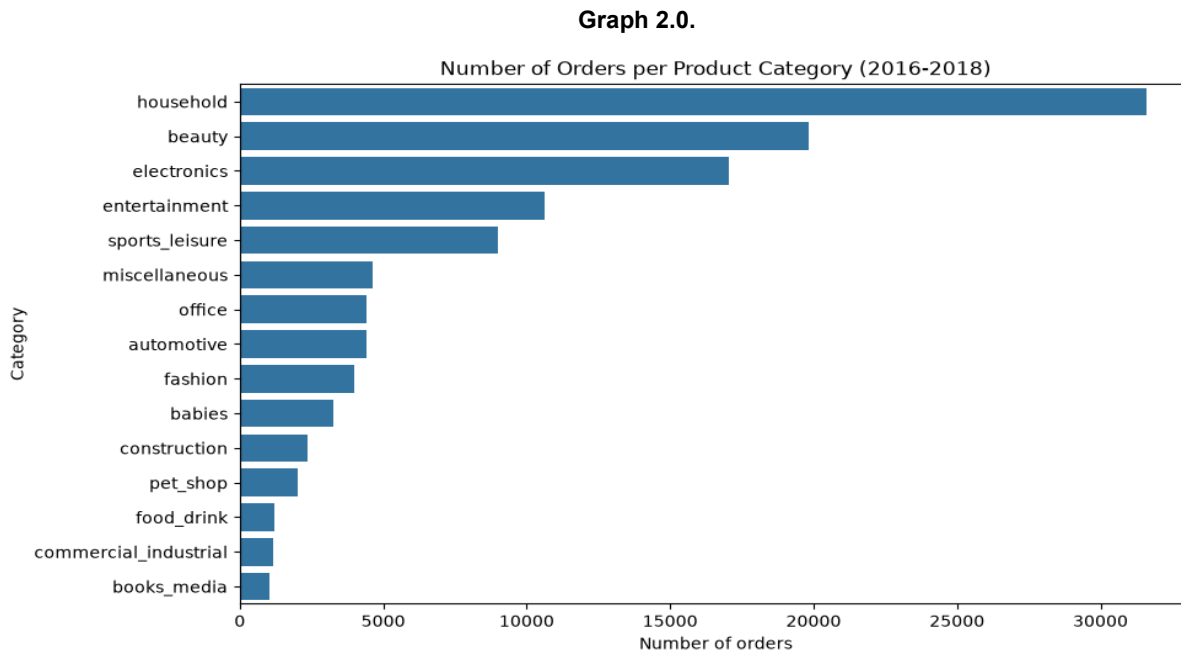
Graph 1.9.



The line plot indicates a spike in delivery between -20 and 0 days. Thus, early shipping is prevalent in the business. On the other hand, there are fewer delays than before-due-date arrivals.

Order quantity per product category

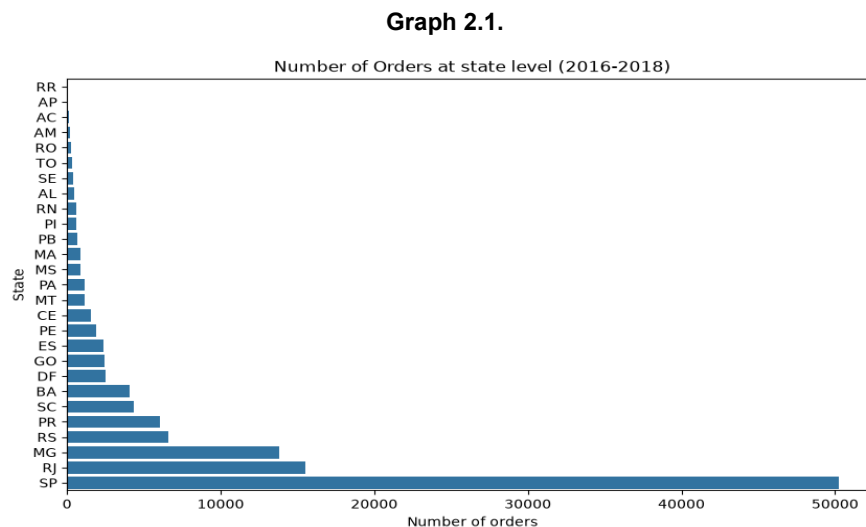
This section analyses the product order quantity by product category. To handle the excessive number of categories(71 categories), the existing categories were abstracted to create a new category("product_cat"). This category has 15 classes, and they are quantified to detect the order volume in the chart.



Households are the most purchased goods on the Olist platform by a large margin. Beauty, electronics and entertainment categories follow household purchase volume. In contrast, food_drink, commercial_industrial, books, and media were the least purchased categories.

Order volume by state

As seen in the statistical summary section, Sao Paulo has the highest order number, followed by Rio de Janeiro and Minas Gerais. Interestingly, purchases increase significantly in these three states in contrast to the other 23.



Average days to repurchase

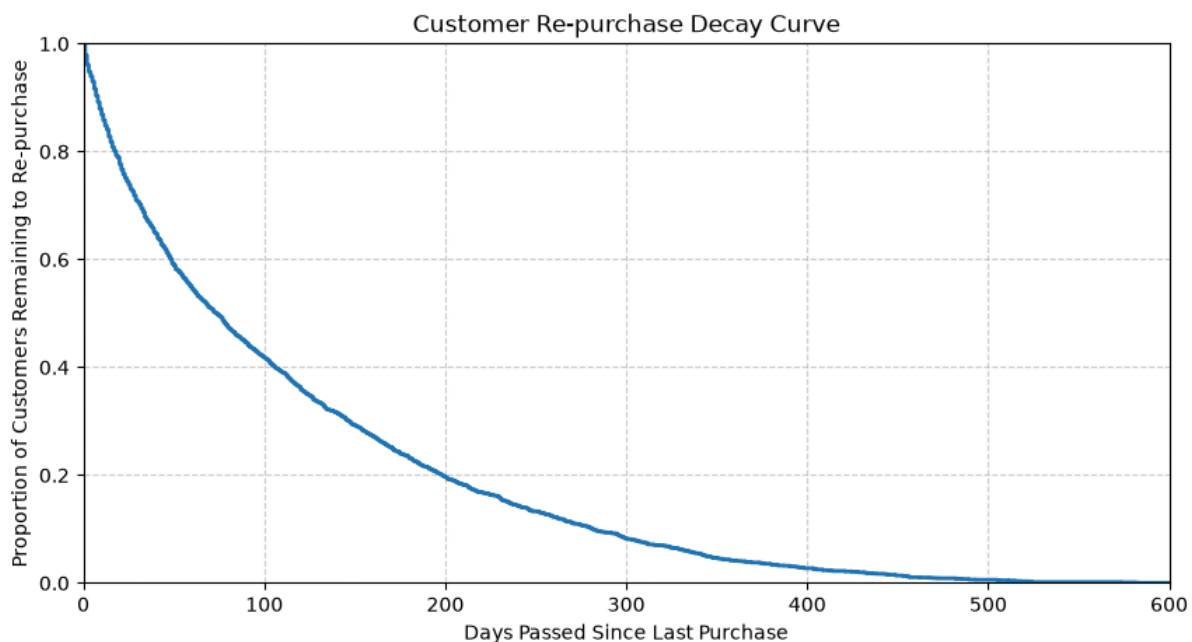
This critical step forms the foundation of the training dataset for the machine learning model, since we will determine the timeframe to distinguish between churned and retained customers.

First, we observe order dates for each customer using the `customer_unique_id`, which holds a unique number for each customer. Then, we group the table by customer id, and use the “diff” to calculate the difference between the first purchase and the second purchase.

Once we have discovered the days to the next payment, we average the days for the following purchase. This allows us to count days for customers that repurchased in different periods several times.

Finally, the data is sorted in ascending order, and the customer’s average dates are counted to create a graph that depicts the average time(in days) in which customers shop on the platform again

Graph 2.2.



Graph 1.2 highlights high retention in the first days; however, it gradually declines over time. We have to choose an intermediate date. If we set the threshold too early, most customers who repurchased will have already done so. Conversely, if we set the cut-off point after 200 days, most clients will have stopped purchasing.

Based on this logic, we will set three different threshold days to proceed with the study to assess which threshold day generates better results in our machine learning models.

The limit days to identify a churned customer would be:

- 90 days
- 120 days
- 150 days

Data Preparation

In this stage, the data will pass through data quality improvement processes, where it will be cleaned, and new features will be introduced. During this process, null values are removed, and some are replaced with 0. This eliminates distortion and avoids errors in machine learning.

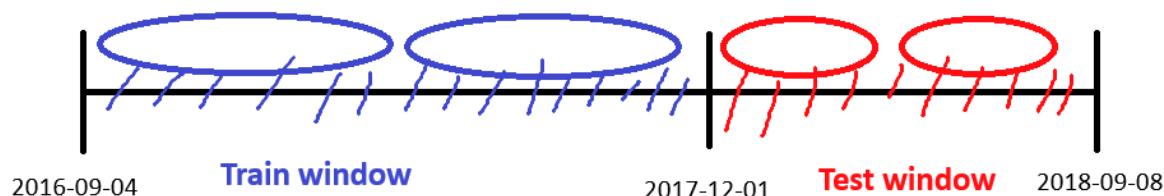
Data cleaning

To begin, null values for critical features such as “price”, “product_id”, “order_approved_at”, and “payment_type” are removed. The reason behind this is that we can not detect churn without order details. Moreover, without the order approval date, it’s impossible to confirm if the order took place. For the next step, we will remove columns(see appendix) that will not be used in the machine learning model, including the comment messages with a high rate of null values we found in the EDA stage.

Finally, to ensure that any null values are removed, we replace null values with 0 using the “fillna()” function for numerical variables. In contrast, for categorical variables, we replace null rows with the word “unknown”. Based on this logic, we will set three different threshold days to proceed with the study to assess which threshold

Data splitting criteria

Before generating new features and the label (churn), the split criteria for training and test datasets must be defined.



The training and test datasets will contain different time periods to avoid data leakage. The training period starts on September 4th 2016, and finishes on November 28th 2017, while the testing period starts on December 1st 2017 and ends on September 9th 2018. Upon splitting the training and test periods, various training datasets for each threshold date will be constructed.

The training and testing data for 90-day analysis will have two terms. Each term comprises 180 days. The prior 90 days are used to identify customers, while in the remaining 90 days each customer purchase is verified. If the purchaser paid during the post-90 days, they are considered retained customers; otherwise, they left (churned).

For the other market dates (120 and 150), each contains only 2 terms to keep them within the training period. The logic for the term works in the same way as for the 90-day threshold: a total of 240 days to detect churn for the 120-day limit, and 300 days for 150 in each term.

Conversely, concerning the 120-day and 150-day test datasets, the former has two terms, while the latter has only one period, as there were not enough dates to fit 600 days.

Feature engineering

Feature creation

During our data exploration, we analysed a few business metrics that are essential for maintaining customer relations. For example, the delivery time and freight cost are two of them. If the packages arrive after the expected date, the buyer might get frustrated. Moreover, if shipping costs are high, the individual could abandon the shopping cart or pay one time and never return.

At the same time, sales volume by product category merits further investigation. If we detect a category that implies a higher churn probability, the business can apply additional strategies such as cross-selling.

We generated additional columns created from customer state, where we divide customer location into high-, medium-, and low-income states, to analyse if average income for each state provides any valuable information for churn.

Thus, we have created new features called "freight_to_price_ratio", "freight_to_weight_ratio", "date_diff", "product_cat", high_income_state, medium_income_state, and low_income_state

Once the previous features were created, the entire Olist data was split, and the churn column was introduced using the criteria explained above. During the feature creation process, categorical columns such as "seller_state", IDs, and zip codes are excluded from the table. The churn feature is a binary variable, where retained customers are encoded as 1, and churned ones as 0.

Feature transformation

Feature transformation will be applied to the categorical variables for the training and test datasets. These variables are converted to binary variables using a scikit-learn class called "OneHotEncoder". This class helps create columns per group in that category.

For instance, suppose we have a categorical feature that classifies animal taxonomy: mammals, birds, reptiles, amphibians, and fish. A one-hot encoder generates 5 binary columns for each class. Here, the numerical variables are not transformed to preserve the mathematical significance between numbers. In other words, machine learning models should understand the distance between numbers, but if we generate a one-hot column for each number or a group of numbers, the model will not understand their relationship.

Therefore, when applying the one-hot encoder class, we indicate that it should only transform categorical features and pass numerical ones without turning them into 1s and 0s.

Another important point in the feature transformation stage is that we indicate the class to drop one of the binary attributes generated using a single categorical variable. This is pivotal to avoid multicollinearity.

Attributes transformed through one-hot encoding are listed below:

- "order_status"
- "payment_type"
- "product_cat"

Train-test split

After passing through several data processing stages, we can finally enter the final split dataset to be trained by machine learning models.

Before dividing our data into train and test, it is critical to understand the reasoning behind the task. The machine learning model will learn patterns from the training dataset and draw conclusions from it. If we use that same training dataset to test how well the model has learned, it has just memorised the training set, and the prediction will not be reliable. When you try to predict with new information, the model will not be able to draw accurate conclusions. Now that the fundamentals are comprehended, we will proceed with splitting.

For the training encoded table, it is separated into X_train (features) and y_train (target). The target variable contains the churn class. Zero if the customer has churned and 1 if retained. The same method is applied for separating the test dataset into X_valtest and y_valtest.

At this stage, we drop columns that are no longer useful for model training when setting X and y_train. These are the categories used for one-hot encoding and ID columns if they remained.

One problem left was to handle the imbalanced target feature. The churn column presented serious imbalance, where more than 98% of customers belonged to the churned group instead of the retained one.

To tackle this fact, we used SMOTE(synthetic minority oversampling technique), that draw artificial points for the class with few observations to balance out the data.

Data Modeling

Since we have the target value "churn" and it is known, we will use supervised classification models such as Logistic Regression, Decision Tree, Random Forest, and XGBoost. The objective of using these machine learning models is to determine which characteristics are closely related to churn, and which window days (90, 120, 150) have the best predictions. Additionally, the performance of each classifier will be assessed, allowing us to detect the threshold day and the model with accurate predictions.

Hypothesis testing

Feature relations to churn class

Although the majority of features contain low correlation with the churn column(value 1), the highest positive correlation is shown by the sport and leisure category(see the table in the appendix). Therefore, the first hypothesis argument would be “the sport-leisure category increases the probability of customer retention”.

Null hypothesis1 (H1): the sport-leisure category does not increase retention rate

The best model

Another fact that we want to analyse is which of these four models performs more accurately and returns a reliable result. Our second hypothesis is that, given the merits of the XGBoost algorithm that will be explained later, this model would provide better outcomes for the 90-day threshold.

Null Hypothesis2 (H2): The XGBoost algorithm does not obtain the best result for 90 days

Methodoly

Logistic Regression models output a probability value called “p-value”, in which we can verify if the result was not a mere coincidence. However, tree-based models do not provide p-values. Instead, they provide the SHAP(SHapley Additive exPlanations) value, which depicts how each feature contributes to the prediction value positively or negatively.

Having said that, we will set a 95% confidence interval for logistic regression, whereas for tree-based models, the contribution to the prediction is assessed.

On the other hand, to test the second hypothesis, all F1 scores will be compared to discover the day and the model with a higher rate.

Before analysing results, each machine learning model will be explained along with their benefits and downsides.

Logistic Regression

$$P(Y = 1) = \frac{1}{1 + e^{-z}} \quad (1)$$

$$z = w_0 + w_1 feature1 + w_2 feature 2_{(2)}$$

Logistic Regression is a classification approach that tries to predict the probability of an event happening using its independent variables. The first equation, also known as the sigmoid, shows how it calculates the probability of an event occurring. What makes this regression unique is that the sigmoid function ensures that numbers range between 0 and 1. The letter Z represents a linear function, as seen in the second equation; it is composed by multiplying each variable by its corresponding weight.

One of the benefits of logistic regression is that we can compute odds ratios. They indicate us how many times more likely an event is to occur than not occurring, and they can be calculated as follows.

$$odds\ ratio = e^{w_i}$$

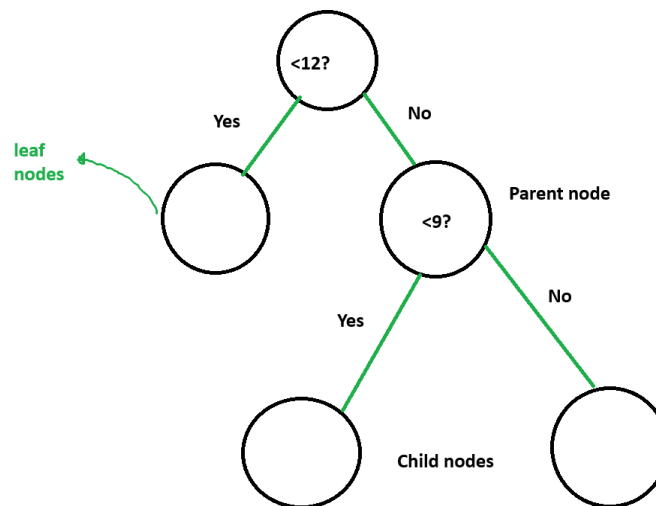
Another advantage is that it produces a p-value and weight that make interpretation easier. We can assess each feature's accuracy and how much it contributed to the final result.

Despite its benefits, the model has downsides. When the model faces multicollinearity, it struggles to assign the correct weights to each feature, decreasing the model's reliability. Logistic Regression is very sensitive to multicollinearity,

Another drawback is that this is a linear classifier, and its classification is based on a linear boundary. Therefore, it can not separate classes with a non-linear relationship.

Decision Tree

Contrary to Logistic Regression, a Decision Tree is a non-linear classifier that distinguishes one class from the other by asking questions. Once questions are made and characteristics are classified, the model calculates an impurity rate using Gini, Entropy or Log-loss criteria. Then, the model computes information gain(IG), which is parent impurity minus child impurity. IG is simulated for different possible questions, and finally the question that provides the greater information gain is selected.



One of the merits of decision trees is that they can approach non-linear problems. Another point is that they are less affected by multicollinearity, as they choose the feature that provides more information gain when asking a question.

A negative side of this type of model is that they are easily overfitted, as it makes decisions using a single tree. If they make too many questions, they will be memorising the data and not learning from it, and therefore, it is essential to set max depth to prevent overfitting. At the same time, they are very sensitive to data, and if a small feature changes, they provide completely different outcomes.

Random Forest

Random Forest compensates for decision tree data sensitivity by using the Ensemble Learning Method. This method consists of training different trees simultaneously. We will explain different phases of random forest in the next sentences.

First, the model builds bootstrapped datasets of similar size to the original dataset. This dataset is made by applying random sampling with repetition. Subsequently, decision trees are trained by selecting a random subset of features for each tree. Once decision trees are trained, the model takes a new data point and starts voting. For example, if there are 5 trained trees and 4 of them vote for class 1, the model will classify it as class 1.

Although we have seen how powerful the ensemble learning method of random forest is, the tremendous number of trees makes it difficult to analyse why the model took the specific decision criteria.

XGBoost

XGBoost is the acronym for extreme gradient boosting, and it's used for large-dataset predictions. This model not only maximises computational efficiency, but also makes continuous tree

improvements, predicting the error produced in the previous tree. The XGBoost model penalises model complexity to avoid overfitting.

Evaluating Results

Goodness of fit testing(H1)

To study whether we can reject the null hypothesis that the sport-leisure category does not imply a higher retention, we will explore results generated from applying logistic regression and the other three tree-based models.

We intended to generate a statistical summary based on maximum likelihood estimation to assess the coefficients' contribution to the prediction. However, given the high dimensionality of the dataset, we could not remove the strict multicollinearity and matrix singularity even after trying to remove high correlation between variables during the one-hot encoding stage. Consequently, we could not generate a summary for our logistic regression model, and thus the results are not reliable for rejecting our null hypothesis. Fortunately, each coefficient and odds ratio is still available in the appendix to assess how they contribute to the model.

For SHAP values, we can visualise them in the appendix as Figure 1.1. Starting with features describing the customer residence and their living state, average income, it can be understood that for high, medium, and low income, their distribution along the graph is similar and is volatile. This means that their contribution to the model can be positive or negative, which, in turn, does not show a clear contribution. On the contrary, other features placed on the bottom side of the figure show a negative relationship. For payments in credit card, it shows that customers choosing a credit card as a payment method are slightly more likely to churn(approximates to 0, churn). Sport-leisure, office, miscellaneous, fashion, and unknown categories also contribute negatively to 1(retained customers). Therefore, clients who shop in these categories are prone to churn as well.

Finally, due to the low reliability of the logistic regression model caused by multicollinearity, and the fact that the sport-leisure category contributed negatively to the model shown by the SHAP plot, we fail to reject our null hypothesis.

Evaluating the second hypothesis

As you can note from the Results section in the appendix, the overall training F1 score shows a near-perfect prediction, while when looking to tests f1 score, the prediction rate drops significantly. This clearly shows a severe overfitting pattern.

We tried several predictions using different parameters, such as `n_estimators`, `max_depth`, and learning rate, to control overfitting. Nevertheless, even after reducing the hyperparameter complexity, we could not prevent the models from memorising noise in the dataset. From this

result, we conclude that the difficulty in obtaining a generalisable model prediction has nothing to do with model or hyperparameter selection, but rather with feature limitation. Most features describe customer payment transactions, satisfaction scores, and payment location, but they lack behavioural navigation information data

When arguing about the model that best predicts retention and churn rates, XGBoost gets a higher F1 score for all different threshold days, contrary to Logistic Regression that scored the lowest prediction. As we explained before, XGBoost tries to predict the error from the previous tree to fix it, while elevated multicollinearity harmed the logistic regression model's credibility.

In conclusion, the model shows higher predictability for the XGBoost model, specifically for the 90- and 120-day windows, but since we have a severe overfitting problem, our model lacks sufficient reliability to reject the null hypothesis that XGBoost for 90 days does not have the best prediction.

Business conclusions and Insight

Although our attempt to reduce model complexity by applying penalisation and reducing tree depth, we could not overcome severe overfitting and multicollinearity, producing a low reliable results. However, this outcome generated new business value objectives to improve with other metrics we have seen in the EDA step.

One point we suggest is improving freight-to-weight ratio efficiency by packing small products with larger ones to reduce logistics costs. At the same time, we encourage cross-promoting strategies to sell least-demanded categories with the most-demanded ones to not only increase sales, but also reduce churn per category.

Another point we recommend is introducing our customer behaviour data that tracks their activities on the platform to improve explainability and reduce overfitting.

Bibliography

<https://www.mdpi.com/0718-1876/19/2/76>

<https://www.monks.com/articles/amazon-1p-vs-3p-vs-hybrid-how-choose-right-supply-method>

<https://true-global-ec.com/ec-info-english/post-4852/>

<https://www.practicalecommerce.com/ecommerce-in-brazil-growth-despite-hurdles>

<https://www.sellerassistant.app/blog/amazon-1p-vs-3p/>

https://mpra.ub.uni-muenchen.de/107298/1/MPRA_paper_107298.pdf

<https://segmentify.com/blog/brief-look-history-ecommerce/>

<https://www.paddle.com/resources/customer-churn>

<https://www.ibm.com/think/topics/customer-churn>

Appendix

GitHub repository: <https://github.com/SilvanaKoga/tfm-uned-churn>

Quantitative variables

```
in_list = ['payment_sequential', 'payment_installments', 'review_score', 'price',  
'freight_value', 'payment_value', 'product_name_lenght',  
'product_description_lenght', 'product_photos_qty', 'product_weight_g',  
'product_length_cm', 'product_height_cm', 'product_width_cm']  
  
olist_df[in_list].describe().to_excel("Numerical_Variables_summary.xlsx")
```

Table 1.1

	price	freight_value	payment_value	product_name_lenght
count	118310	118310	119140	116601
mean	120.6466034	20.03238653	172.7351352	48.76749771
std	184.1096908	15.83685044	267.7760767	10.03354018
min	0.85	0	0	5
25%	39.9	13.08	60.85	42
50%	74.9	16.28	108.16	52
75%	134.9	21.18	189.24	57
max	6735	409.68	13664.08	76

Table 1.2

	product_description_lenght	product_photos_qty	product_weight_g	product_length_cm	product_height_cm	product_width_cm
count	116601	116601	118290	118290	118290	118290
mean	785.9678219	2.205161191	2112.25074	30.26514498	16.61970581	23.07479922
std	652.5841208	1.717451875	3786.695111	16.18936729	13.45358398	11.74913915
min	4	1	0	7	2	6
25%	346	1	300	18	8	15
50%	600	1	700	25	13	20
75%	983	3	1800	38	20	30
max	3992	20	40425	105	105	118

Table 1.3

	payment_sequential	payment_installments	review_score
count	119140	119140	118146
mean	1.094737284	2.941245593	4.015582415
std	0.7301409934	2.777847747	1.400435574
min	1	0	1
25%	1	1	4
50%	1	2	5
75%	1	4	5
max	29	24	5

Graph 1.1 (code)

Price- code block 1

```
plt.figure(figsize= (10,4))
sns.boxplot(data= olist_df, x = "price")

plt.title("Price Distribution (2016-2018)")
plt.xlabel("price")

plt.show()
```

Price - code block 2

```
# Applying xlim to price distribution to make the box visible

plt.figure(figsize= (10,4))
sns.boxplot(data= olist_df, x = "price")

plt.xlim((0,500))
plt.title("Price Distribution (2016-2018)")
plt.xlabel("price")

plt.show()
```

Graph 1.2

Freight price distribution- code block

```
plt.figure(figsize= (10,4))
sns.boxplot(data= olist_df, x = "freight_value")

plt.title("Freight Price Distribution (2016-2018)")
plt.xlabel("freight value")

plt.show()
```

Graph 1.3

Total paid value distribution - code block

```
# Applying xlim to price distribution to make the box visible

plt.figure(figsize= (10,4))
sns.boxplot(data= olist_df, x = "payment_value")

plt.xlim((0,500))
plt.title("Total-Paid Value Distribution (2016-2018)")
plt.xlabel("Payment value")

plt.show()
```

Graph 1.4

Photo quantities used - code block

```
# Compare ranges of photo quantity

plt.figure(figsize= (10,4))
sns.histplot(data = olist_df, x = "product_photos_qty", binwidth= 1, binrange=
(0,20))

plt.title("Picture Quantities Used (2016-2018)")
plt.xlabel("photo quantity")

plt.show()
```

Graph 1.5

Weight distribution - code block

```
plt.figure(figsize= (10,4))
sns.boxplot(data= olist_df, x = "product_weight_g")

plt.xlim((0,5000))
plt.title("Product Weight Distribution(2016-2018)")
plt.xlabel("weight (g)")

plt.show()
```

Qualitative variables

```
categorical_list = [ 'customer_city', 'customer_state', 'order_status',
                    'payment_type', 'review_comment_title',
                    'review_comment_message',
                    'product_category_name', 'seller_city', 'seller_state',
                    'product_category_name_english' ]

olist_df[categorical_list].describe(include= ["object",
"category"]).to_excel("Categorical_Variables_summary.xlsx")
```

Table 1.4

列 1	customer_c ity	customer_sta te	payment_t ype	review_commen t_title	review_comment_ message
cou nt	119143	119143	119140	13989	50245
uniq	4119	27	5	4527	36159

ue					
top	sao paulo	SP	credit_card	Recomendo	Muito bom
freq	18875	50265	87776	494	259

Table 1.5

列 1	product_category_name	seller_city	seller_state	product_category_name_english
count	116601	118310	118310	116576
unique	73	611	23	71
top	cama_mesa_banheiro	sao paulo	SP	bed_bath_table
freq	11988	29293	84377	11988

Business Metrics

Graph 1.6

Freight-to-price ratio distribution - code block

```
plt.figure(figsize= (10,4))
sns.histplot(data = olist_df, x = "freight_to_price_ratio", binwidth= 0.1,
binrange= (0,2))

plt.xlim((0,2))
plt.xticks(np.arange(0.0,2,0.2))
plt.title("Freight_to_price_ratio distribution (2016-2018)")
plt.xlabel("ratio")

plt.show()
```

Graph 1.7

freight cost efficiency - code block

```
from matplotlib.ticker import ScalarFormatter

# Plotting a regression graph
fig, ax = plt.subplots(figsize = (9, 5.5))
sns.regplot(data = olist_df, x = "product_weight_g", y =
"freight_to_weight_ratio",
```

```

        logx = True, scatter_kws={"alpha": 0.3, "color": "blue"}, line_kws=
{"color": "red", "linewidth": 2}, ax= ax)

for axis in [ax.xaxis, ax.yaxis]:
    axis.set_major_formatter(ScalarFormatter())
    axis.set_minor_formatter(ScalarFormatter())
ax.set_xscale("log")
ax.set_yscale("log")
plt.title("Freight Cost efficiency (2016-2018)")
plt.xlabel("Weight (g- log scale)")
plt.ylabel("freight to weight ratio")

plt.show()

```

Graph 1.8

Order volume per quarter - code block

```

# Build a dataframe tha containg two attributes, purchase_date, and its
counts(times ordered)
purchases_timeline =
olist_df["order_purchase_timestamp"].value_counts().sort_index().reset_index()
purchases_timeline.columns = ["purchase_date", "counts"]

# Ensure that purchase_date is a datetime
purchases_timeline["purchase_date"] =
pd.to_datetime(purchases_timeline["purchase_date"])

# Adjust purchase date to have quarterly date period
purchases_timeline["quarter"] =
purchases_timeline["purchase_date"].dt.to_period("Q")

# Group the column by quarter, and aggregate counts
purchases_per_quarter =
purchases_timeline.groupby("quarter")["counts"].sum().reset_index()

purchases_timeline["quarter"] = purchases_timeline["quarter"].astype(str)

plt.figure(figsize= (10,4))
sns.lineplot(data = purchases_timeline, x = "quarter", y = "counts")

plt.title("Order Volumn per Quarter (2016-2018)")
plt.xlabel("Quarters")
plt.ylabel("Order Counts")

plt.show()

```

Graph 1.9

Delays in days - code block

```
olist_df["order_estimated_delivery_date"] =  
pd.to_datetime(olist_df["order_estimated_delivery_date"])  
olist_df["order_delivered_customer_date"] =  
pd.to_datetime(olist_df["order_delivered_customer_date"])  
  
olist_df["dates_diff"] = (olist_df["order_delivered_customer_date"] -  
olist_df["order_estimated_delivery_date"]).dt.days  
  
olist_df["dates_diff"]  
  
# Negative values means early arrival, while positive ones are delays  
delay_counts = olist_df["dates_diff"].value_counts().sort_index().reset_index()  
delay_counts.columns = ["Days Past Due", "Number of delays"]  
  
plt.figure(figsize= (10,4))  
sns.lineplot(data = delay_counts, x = "Days Past Due", y = "Number of delays")  
  
plt.title("Delays in days (2016-2018)")  
plt.xlabel("Days Past Due")  
plt.ylabel("Number of delays")  
  
plt.show()
```

Graph 2.0

Number of orders per product category - code block

```
# Plotting orders by product category  
olist_df["product_cat"] =  
olist_df.product_category_name_english.map(product_categories_map)  
order_num_per_cat = olist_df["product_cat"].value_counts().sort_values(ascending=  
False).reset_index()  
  
order_num_per_cat.columns = ["Category", "Number of orders"]  
  
plt.figure(figsize= (10,6))  
sns.barplot(data = order_num_per_cat, x = "Number of orders", y = "Category")  
  
plt.title("Number of Orders per Product Category (2016-2018)")  
plt.xlabel("Number of orders")  
plt.ylabel("Category")  
  
plt.show()
```

Graph 2.1

Number of orders at state level - code block

```
# Order volume by State
```

```

Order_num_state =
olist_df["customer_state"].value_counts().sort_values().reset_index()

plt.figure(figsize= (10,6))
sns.barplot(data = Order_num_state, x = "count", y = "customer_state")

plt.title("Number of Orders at state level (2016-2018)")
plt.xlabel("Number of orders")
plt.ylabel("State")

plt.show()

```

Graph 2.2

Customer Re-purchase Decay Curve - code block

```

plt.figure(figsize=(10, 5))

# complementary=True flips the line to slope downwards from 1.0 to 0.0
sns.ecdfplot(data=cph_copy_filtered, x="avg_days_to_re_purchase",
complementary=True, lw=2)

# Set x-axis limit (expanded to 180 days to see where it flattens near the
bottom)
plt.xlim(0, 600)
plt.grid(True, linestyle="--", alpha=0.6)

plt.xlabel("Days Passed Since Last Purchase")
plt.ylabel("Proportion of Customers Remaining to Re-purchase")
plt.title("Customer Re-purchase Decay Curve")
plt.show()

```

Data Preparation

Data cleaning

After cleaning

Features	null_values
customer_id	0
customer_unique_id	0
customer_zip_code_prefix	0
customer_city	0
customer_state	0
order_id	0
order_status	0

order_purchase_timestamp	0
order_approved_at	0
order_estimated_delivery_date	0
order_item_id	0
product_id	0
seller_id	0
shipping_limit_date	0
price	0
freight_value	0
payment_sequential	0
payment_type	0
payment_installments	0
payment_value	0
review_score	0
product_photos_qty	0
seller_zip_code_prefix	0
seller_city	0
seller_state	0
product_category_name_english	0
freight_to_price_ratio	0
freight_to_weight_ratio	0
dates_diff	0
product_cat	0

Feature creation

“freight_to_price_ratio“, “freight_to_weight_ratio“, “date_diff“, and “product_cat“, churn column, state income level

```
# Adding a new column that compares the freight cost proportion to the price
olist_df["freight_to_price_ratio"] = olist_df["freight_value"]/olist_df["price"]
```

```
olist_df["freight_to_weight_ratio"] =
olist_df["freight_value"]/olist_df["product_weight_g"]
```

```
olist_df["order_estimated_delivery_date"] =
pd.to_datetime(olist_df["order_estimated_delivery_date"])
olist_df["order_delivered_customer_date"] =
pd.to_datetime(olist_df["order_delivered_customer_date"])

olist_df["dates_diff"] = (olist_df["order_delivered_customer_date"] -
```

```
olist_df["order_estimated_delivery_date"]).dt.days
```

```
olist_df["dates_diff"]
```

```
olist_df["product_cat"] =  
olist_df.product_category_name_english.map(product_categories_map)
```

```
HIGH_INCOME = ["SP", "RJ", "AM", "SE"]  
MID_INCOME = ["PI", "PB", "MS", "ES", "BA", "AL", "PE", "RN", "CE"]  
LOW_INCOME = ["RR", "RO", "MT", "GO", "TO", "RS", "SC", "PR", "MA", "MG", "AC",  
"PA", "AP", "DF"]  
  
prov_income_map = {  
    **{i:"high" for i in HIGH_INCOME},  
    **{i:"mid" for i in MID_INCOME},  
    **{i:"low" for i in LOW_INCOME}  
}  
prov_income_map
```

```
Data_train_90_df["state_income_level"] =  
Data_train_90_df.customer_state.map(prov_income_map)  
  
Data_train_90_df["state_income_level_high"] =  
(Data_train_90_df["state_income_level"] == "high").astype(int)  
Data_train_90_df["state_income_level_mid"] =  
(Data_train_90_df["state_income_level"] == "mid").astype(int)  
Data_train_90_df["state_income_level_low"] =  
(Data_train_90_df["state_income_level"] == "low").astype(int)
```

feature transformation

One-hot encoding

90-days

```
target_features = ["order_status", "payment_type", "product_cat"]  
  
col_trans = ColumnTransformer(  
    transformers=[("cat", OneHotEncoder(sparse_output=False, drop="first",  
handle_unknown="ignore"), target_features)], remainder="passthrough"  
)  
col_trans.fit(Data_train_90_df)
```

120-days

```
target_features = ["order_status", "payment_type", "product_cat"]

col_trans = ColumnTransformer(
    transformers=[("cat", OneHotEncoder(sparse_output= False, drop="first",
handle_unknown="ignore"), target_features)], remainder="passthrough"
)
col_trans.fit(Data_train_120_df)
```

150-days

```
target_features = ["order_status", "payment_type", "product_cat"]

col_trans = ColumnTransformer(
    transformers=[("cat", OneHotEncoder(sparse_output= False, drop= "first",
handle_unknown="ignore"), target_features)], remainder="passthrough"
)
col_trans.fit(Data_train_150_df)
```

Train-test split

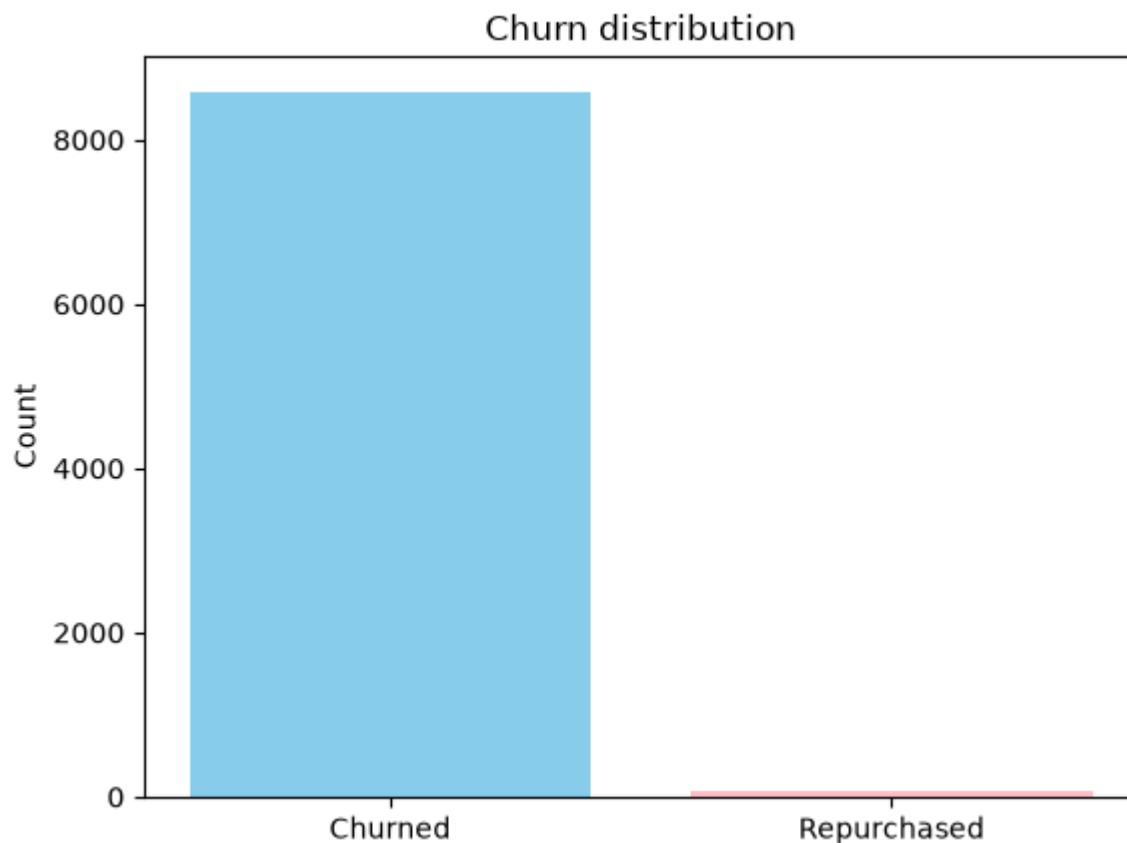
```
X_train = encoded_90_df.drop(columns = ['churn_90', "customer_unique_id",
"customer_city", "customer_state", "state_income_level"])

y_train = encoded_90_df['churn_90'].fillna(0).astype(int)
```

```
X_valtest = encoded_90_test_df.drop(columns = ['churn_90', "customer_unique_id",
"customer_city", "customer_state", "state_income_level"])

y_valtest = encoded_90_test_df['churn_90'].fillna(0).astype(int)
```

```
# Here. we see how much imbalance we have for out target feature
plt.bar(y_train.value_counts().index, y_train.value_counts().values,
color=['skyblue','pink'])
plt.xticks([0,1], ["Churned", "Repurchased"])
plt.ylabel("Count")
plt.title("Churn distribution")
plt.show()
```



```
from sklearn.model_selection import train_test_split

X_val, X_test, y_val, y_test = train_test_split(X_valtest, y_valtest,
test_size= 0.5, random_state = 42)
X_val = X_val.astype(int)
X_test = X_test.astype(int)
```

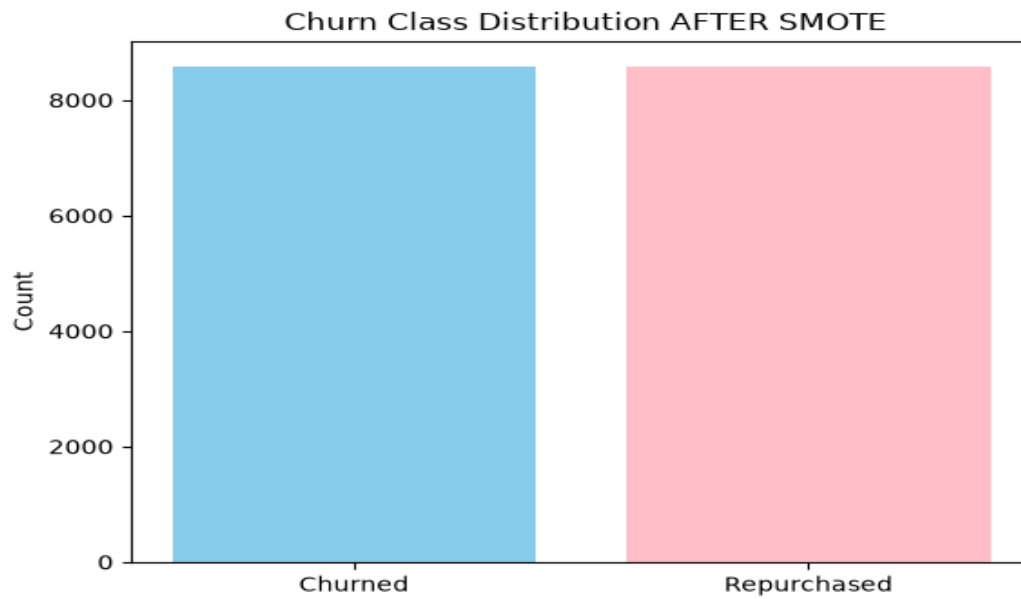
After applying SMOTE

```
from imblearn.over_sampling import SMOTE

# We used SMOTE(Synthetic minority oversampling technique) to deal with
imbalance by plotting artificial points for the minority class

smote = SMOTE(sampling_strategy="minority", random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)

plt.bar(y_train_sm.value_counts().index, y_train_sm.value_counts().values,
        color=["skyblue", "pink"])
plt.xticks([0,1], ["Churned", "Repurchased"])
plt.ylabel("Count")
plt.title(" Churn Class Distribution AFTER SMOTE")
plt.show()
```



Evaluating Results

Results for coefficients, odds, and SHAP

90 days

const	Feature	Beta	Odds_Ratio	constant	Feature (19-35)	(19-35)	Odds_Ratio (19-35)
0	state_income_level_high	-4.1421	0.0159	19	cat__order_status_invoiced	-0.4562	0.6337
1	state_income_level_low	-3.7404	0.0237	20	cat__order_status_canceled	-0.3918	0.6758
2	state_income_level_mid	-2.4567	0.0857	21	cat__payment_type_voucher	-0.3697	0.6909
3	cat__product_cat_entertainment	-1.7254	0.1781	22	cat__order_status_processing	-0.3687	0.6916
4	cat__product_cat_household	-1.6687	0.1885	23	freight_to_price_ratio	-0.3616	0.6966
5	cat__product_cat_beauty	-1.3855	0.2502	24	cat__payment_type_debit_card	-0.2968	0.7432
6	cat__product_cat_electronics	-1.3836	0.2507	25	cat__product_cat_construction	-0.2661	0.7664
7	cat__product_cat_miscellaneous	-1.0234	0.3594	26	cat__product_cat_food_drink	-0.2482	0.7802

8	cat__product_cat_office	-0.9653	0.3809	27	cat__payment_type_credit_card	-0.2106	0.8101
9	cat__product_cat_sports_leisure	-0.8147	0.4428	28	payment_installments	0.1847	1.2028
10	cat__product_cat_fashion	-0.7981	0.4502	29	payment_sequential	-0.173	0.8411
11	cat__product_cat_unknown	-0.792	0.4529	30	price	-0.1398	0.8696
12	cat__product_cat_babies	-0.7047	0.4942	31	cat__order_status_unavailable	-0.1207	0.8863
13	cat__product_cat_books_media	-0.6474	0.5234	32	review_score	-0.0589	0.9428
14	cat__order_status_shipped	-0.5847	0.5573	33	payment_value	-0.0508	0.9505
15	cat__product_cat_pet_shop	-0.581	0.5593	34	freight_to_weight_ratio	-0.0285	0.9719
16	cat__order_status_delivered	-0.5509	0.5764	35	dates_diff	0.0072	1.0072
17	cat__product_cat_commercial_industrial	-0.5033	0.6045				
18	product_photos_qty	-0.4992	0.607				

120 days

const	Feature	Beta	Odds_Ratio	const	Feature (19-35)	Beta 19-35()	odds
0	state_income_level_high	-4.1421	0.0159	19	cat__order_status_invoiced	-0.4562	0.6337
1	state_income_level_low	-3.7404	0.0237	20	cat__order_status_cancelled	-0.3918	0.6758
2	state_income_level_mid	-2.4567	0.0857	21	cat__payment_type_voucher	-0.3697	0.6909
3	cat__product_cat_entertainment	-1.7254	0.1781	22	cat__order_status_processing	-0.3687	0.6916
4	cat__product_cat_household	-1.6687	0.1885	23	freight_to_price_ratio	-0.3616	0.6966
5	cat__product_cat_beauty	-1.3855	0.2502	24	cat__payment_type_debit_card	-0.2968	0.7432
6	cat__product_cat_electronics	-1.3836	0.2507	25	cat__product_cat_construction	-0.2661	0.7664

7	cat__product_cat_miscellaneous	-1.0234	0.3594	26	cat__product_cat_food_drink	-0.2482	0.7802
8	cat__product_cat_office	-0.9653	0.3809	27	cat__payment_type_credit_card	-0.2106	0.8101
9	cat__product_cat_sports_leisure	-0.8147	0.4428	28	payment_installments	0.1847	1.2028
10	cat__product_cat_fashion	-0.7981	0.4502	29	payment_sequential	-0.173	0.8411
11	cat__product_cat_unknown	-0.792	0.4529	30	price	-0.1398	0.8696
12	cat__product_cat_babies	-0.7047	0.4942	31	cat__order_status_unavailable	-0.1207	0.8863
13	cat__product_cat_books_media	-0.6474	0.5234	32	review_score	-0.0589	0.9428
14	cat__order_status_shipped	-0.5847	0.5573	33	payment_value	-0.0508	0.9505
15	cat__product_cat_pet_shop	-0.581	0.5593	34	freight_to_weight_ratio	-0.0285	0.9719
16	cat__order_status_delivered	-0.5509	0.5764	35	dates_diff	0.0072	1.0072
17	cat__product_cat_commercial_industrial	-0.5033	0.6045				
18	product_photos_qty	-0.4992	0.607				

150 days

const	Feature	Beta	Odds_Ratio	const	Feature (19-35)	Beta 19-35()	odds
0	state_income_level_high	-4.1421	0.0159	19	cat__order_status_invoiced	-0.4562	0.6337
1	state_income_level_low	-3.7404	0.0237	20	cat__order_status_cancelled	-0.3918	0.6758
2	state_income_level_mid	-2.4567	0.0857	21	cat__payment_type_voucher	-0.3697	0.6909
3	cat__product_cat_entertainment	-1.7254	0.1781	22	cat__order_status_processing	-0.3687	0.6916
4	cat__product_cat_household	-1.6687	0.1885	23	freight_to_price_ratio	-0.3616	0.6966
5	cat__product_cat_beauty	-1.3855	0.2502	24	cat__payment_type_debit_card	-0.2968	0.7432
6	cat__product_cat_electronics	-1.3836	0.2507	25	cat__product_cat_construction	-0.2661	0.7664
7	cat__product_cat_miscellaneous	-1.0234	0.3594	26	cat__product_cat_food_drink	-0.2482	0.7802

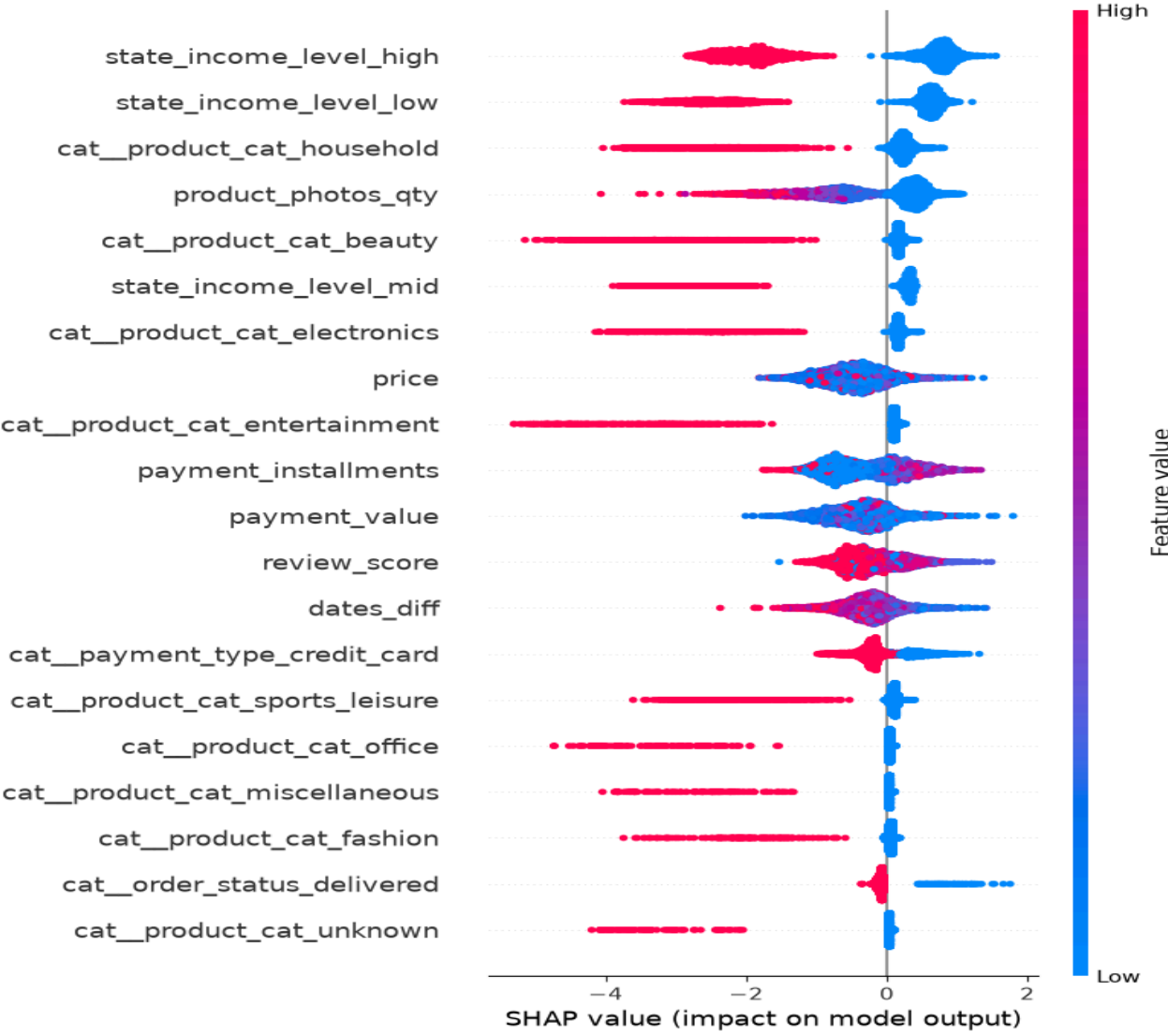
8	cat__product_cat_office	-0.9653	0.3809	27	cat__payment_type_credit_card	-0.2106	0.8101
9	cat__product_cat_sports_leisure	-0.8147	0.4428	28	payment_installments	0.1847	1.2028
10	cat__product_cat_fashion	-0.7981	0.4502	29	payment_sequential	-0.173	0.8411
11	cat__product_cat_unknown	-0.792	0.4529	30	price	-0.1398	0.8696
12	cat__product_cat_babies	-0.7047	0.4942	31	cat__order_status_unavailable	-0.1207	0.8863
13	cat__product_cat_books_media	-0.6474	0.5234	32	review_score	-0.0589	0.9428
14	cat__order_status_shipped	-0.5847	0.5573	33	payment_value	-0.0508	0.9505
15	cat__product_cat_pet_shop	-0.581	0.5593	34	freight_to_weight_ratio	-0.0285	0.9719
16	cat__order_status_delivered	-0.5509	0.5764	35	dates_diff	0.0072	1.0072
17	cat__product_cat_commercial_industrial	-0.5033	0.6045				
18	product_photos_qty	-0.4992	0.607				

Result for each model on different days

90 days

name	Trainsm_f1score	Trainsm_acc	Trainsm_auc	Train_f1score	Train_acc	Train_auc
XGBoost - 100	0.9921516191	0.9921356169	0.9991466635	0.3005181347	0.9844146848	0.925247435
DT - gini	0.9549764379	0.9538040312	0.9800969592	0.1677588467	0.9266912953	0.9636608722
RF - gini_100	0.960183939	0.9586391705	0.9976707966	0.1383495146	0.9180327869	0.9373127923
RF - gini_150	0.9562060105	0.9543283234	0.9968414596	0.1230425056	0.9094897252	0.9307713068
RF - gini_50	0.9547710669	0.9527554468	0.9974057557	0.1213434453	0.9063726622	0.9279359995
DT - entropy	0.9443600563	0.9423861121	0.9597726019	0.109795479	0.9045255137	0.9257614035
LogReg	0.9471056698	0.9481533263	0.9842711524	0.01680672269	0.9594781806	0.6173846741

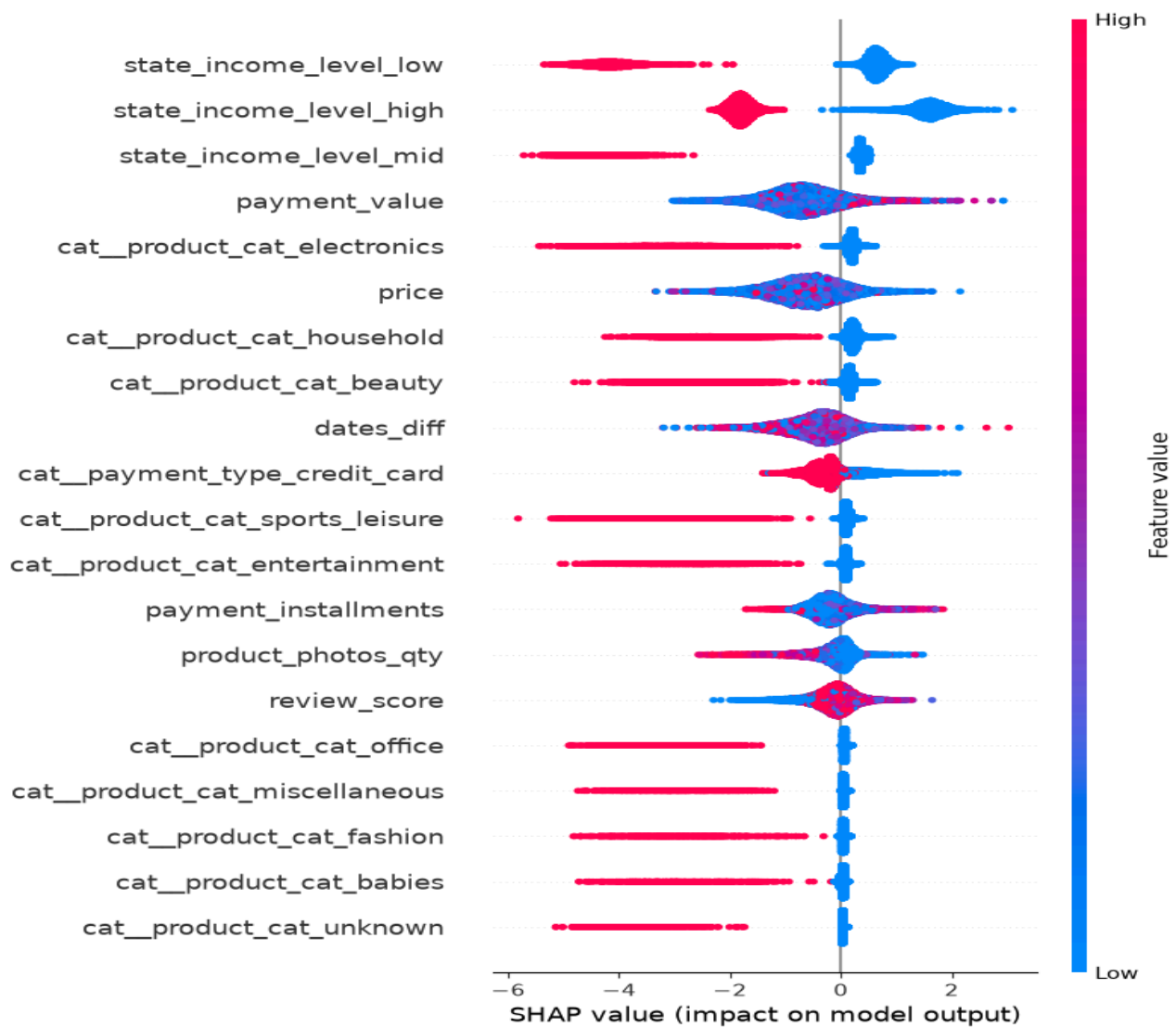
name	Val_f1score	Val_acc	Val_auc	Test_f1score	Test_acc	Test_auc
XGBoost - 100	0.3333333333	0.9833756638	0.9184473707	0.2588235294	0.9854537058	0.9324375889
DT - gini	0.1724137931	0.9224197645	0.9590419993	0.162464986	0.9309628261	0.9684452205
RF - gini_100	0.1405895692	0.9124913415	0.9350269004	0.135770235	0.9235742323	0.9395615056
RF - gini_150	0.1198347107	0.9016393443	0.9270706786	0.1268292683	0.9173401062	0.9339800802
RF - gini_50	0.1190965092	0.9009466636	0.9257419299	0.123853211	0.9117986608	0.9302127797
DT - entropy	0.1171717172	0.8990995151	0.9250531499	0.1013824885	0.9099515124	0.9255367999
LogReg	0.02116402116	0.9572846918	0.6180200017	0.0119047619	0.9616716694	0.6160328547



120-days

name	Trainsm_f1score	Trainsm_acc	Trainsm_auc	Train_f1score	Train_acc	Train_auc
XGBoost - 100	0.9921707286	0.9921568627	0.9994665556	0.3856858847	0.9845333734	0.9592941059
RF - gini_100	0.9278692067	0.9239848197	0.9826028132	0.07005375412	0.8657807143	0.7873106274
RF - gini_50	0.9270667226	0.9230487034	0.9816383607	0.06770388101	0.8635282929	0.77798677
DT - gini	0.917441902	0.9123213156	0.965582169	0.06226155959	0.8462096754	0.8152957868
RF - gini_150	0.9218200522	0.9173181531	0.9818269643	0.06225680934	0.8552443877	0.7847131738
DT - entropy	0.9123143728	0.9075268817	0.964485283	0.06055900621	0.8485872313	0.8204092288
LogReg	0.9111920215	0.9148387097	0.9549131893	0.01731996354	0.9460419951	0.5562647586

name	Val_f1score	Val_acc	Val_auc	Test_f1score	Test_acc	Test_auc
XGBoost - 100	0.4129554656	0.9854840324	0.9587936719	0.359375	0.9835827619	0.959703201
RF - gini_100	0.07520397304	0.8695064571	0.7964972659	0.06512890095	0.8620551579	0.7782454459
RF - gini_50	0.07329105004	0.8683551907	0.788076759	0.06243772833	0.8587016367	0.7680157469
DT - gini	0.06551297899	0.8486334968	0.8263497589	0.05908953874	0.8437859753	0.804616776
RF - gini_150	0.06710963455	0.8594453899	0.7975076699	0.05763141229	0.8510435958	0.7721653345
DT - entropy	0.06637168142	0.8521373511	0.8287682504	0.05494505495	0.8450372892	0.8120692505
LogReg	0.01488372093	0.9469916909	0.5484787076	0.01966041108	0.945092347	0.5638293896



150 days

name	Trainsm_f1sore	Trainsm_acc	Trainsm_auc	Train_f1sore	Train_acc	Train_auc
RF - gini_150	0.9345169783	0.9315793186	0.9915014593	0.09518773136	0.8809490676	0.8411413937
DT - gini	0.9410888407	0.9386329971	0.9822740542	0.1213840045	0.8922209852	0.888654972
RF - gini_100	0.9424307036	0.9400084644	0.9918962255	0.107204611	0.8922209852	0.8394450745
RF - gini_50	0.9421706373	0.9399026592	0.9908972073	0.1096698113	0.894934595	0.8301799773
DT - entropy	0.92978029	0.9263948649	0.977257636	0.09702970297	0.8730865572	0.874425713
XGBoost - 100	0.9884142691	0.9883966989	0.9985549014	0.2642369021	0.9775257445	0.9095693096
LogReg	0.940797335	0.9423361783	0.9750577784	0.009273570325	0.9553993877	0.5784607788

name	Val_f1score	Val_acc	Val_auc	Test_f1score	Test_acc	Test_auc
RF - gini_150	0.01857585139	0.844978299 4	0.505065624 5	0.027397260 27	0.8481051 345	0.5523544 629
DT - gini	0.01853411963	0.857570756 2	0.543224198 9	0.025641025 64	0.8606356 968	0.5505420 676
RF - gini_100	0.01929530201	0.857081728 7	0.505559390 7	0.024431339 51	0.8584352 078	0.5391605 942
RF - gini_50	0.01863354037	0.864783911	0.503519853 3	0.024390243 9	0.8679706 601	0.5424946 862
DT - entropy	0.01853911754	0.838193043 6	0.526265138 3	0.021505376 34	0.8442542 787	0.5442194 199
XGBoost - 100	0.01453488372	0.958554923 9	0.493847382 2	0.014858841 01	0.9594743 276	0.4728968 175
LogReg	0.01889020071	0.949202274	0.518219619	0.011862396 2	0.9490831 296	0.5300925 243

